

DADOS & REPRESENTAÇÃO



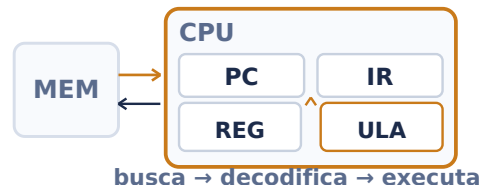
B **5**
8 bits = 1 byte | 1011 0101 = B5h

Codificar números, caracteres e instruções em padrões de bits.

- **Unidades:** 1 byte = 8 bits; 1 KiB = 2^{10} B; 1 MiB = 2^{20} B
- **Base b:** $N = \sum d_i \cdot b^i$
- **Binário ↔ hexadecimal:** 1 dígito hex = 4 bits
- **Sem sinal, n bits:** 0 a $2^n - 1$
- **Complemento de 2:** $-2^{(n-1)}$ a $2^{(n-1)} - 1$; negativo = inverter + 1
- **Overflow com sinal:** operandos de mesmo sinal geram resultado de sinal oposto
- **IEEE 754:** sinal + expoente + fração (mantissa)

Dica: Antes de converter ou somar, fixe a largura em bits. Carry e overflow não significam a mesma coisa.

CPU, ISA & CICLO DE INSTRUÇÃO

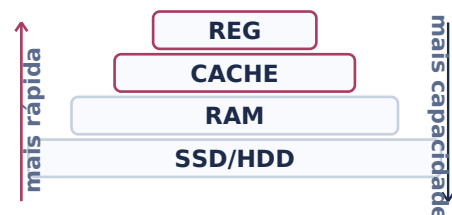


A ISA define o que a máquina executa; a microarquitetura define como.

- **CPU:** controle + ULA + registradores
- **PC/IR:** próxima instrução / instrução atual
- **Ciclo:** busca → decodifica → executa → memória → escrita
- **Instrução:** opcode + operandos; modos localizam os dados
- **ISA:** instruções, registradores, formatos e endereçamento
- **Pipeline:** sobrepõe etapas; hazards estruturais, de dados e de controle
- **Tempo de CPU:** $IC \cdot CPI \cdot T_{clk} = T_{CPU}$

Dica: ISA é a interface; microarquitetura é a implementação. CPUs diferentes podem usar a mesma ISA.

HIERARQUIA DE MEMÓRIA & CACHE



Combinar velocidade, capacidade e custo explorando localidade.

- **Hierarquia:** registradores → cache → RAM → SSD/HDD
- **Localidade temporal:** dado recente tende a ser reutilizado
- **Localidade espacial:** endereços próximos tendem a ser acessados
- **Cache:** transfere blocos/linhas; acesso gera hit ou miss
- **Endereço:** tag | índice | deslocamento
- **AMAT:** $T_{hit} + \text{taxa de miss} \cdot \text{penalidade de miss}$
- **Mapeamento:** direto, associativo por conjunto ou totalmente associativo

Dica: Separe tag, índice e deslocamento usando o tamanho da linha e o número de conjuntos.

E/S, BARRAMENTOS & INTERRUPÇÕES



barramento: dados | endereços | controle

Conectar CPU, memória e periféricos sem ocupar o processador o tempo todo.

- **Barramentos:** dados, endereços e controle
- **MMIO:** registradores do periférico ocupam endereços de memória
- **Polling:** CPU consulta o estado; simples, mas gasta ciclos
- **Interrupção:** salva contexto → ISR → retorno
- **DMA:** move blocos E/S ↔ memória com pouca intervenção da CPU
- **Desempenho:** latência = tempo/operação; banda = dados/tempo

Dica: Use interrupções em eventos esporádicos e DMA em grandes blocos de dados.